Response to the Review Team for OPRE-2025-04-1885

"Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints"

We would like to begin by thanking the Area Editor, the Associate Editor, and the referees for their thoughtful, technically detailed, and constructive feedback. We have taken the reports seriously and revised the manuscript to address the main modeling, queueing, exposition, and numerical concerns raised in the reports and the decision letter. The comments made clear that the original version needed clearer modeling of the memory-coupled control problem, a more convincing explanation of the threshold-based policy design, and more direct numerical support. In the revised manuscript, we have substantially rewritten the model exposition, clarified the queueing objective and memory mechanism, strengthened the theoretical positioning, and expanded the numerical evidence. Before responding point by point, we would like to highlight the most significant changes in the revised manuscript.

- **Reframed the objective and queueing interpretation.** We revised Sections 2–4 to state the exogenous-arrival interpretation more carefully. With a fixed arrival process, any stable policy completes requests at the offered load; the relevant operational question is which arrival rates a policy can stabilize while avoiding eviction-induced restarts and maintaining bounded delay. Accordingly, the paper now uses token-level *effective throughput* for completed decode-token service net of evictions, reports request-level effective completion rate in the experiments, uses the fluid model to identify the stability region and the memory required to sustain it, and plots latency as a function of the arrival rate in Section 6.
- **Rewrote the model and notation.** Section 2 now introduces prefill, decode, batching, KV-cache growth, the memory constraint, and the policy space narratively. The main notation is collected in the Notation Summary appendix, and the stage notation is separated from prompt lengths and output lengths. We also clarified that the memory constraint applies to the KV caches of all in-system requests retained on the GPU, not merely to requests in the current batch.
- **Clarified the time model and its scope.** We expanded the discussion around Equation (1), explaining how the linear memory-dependent iteration-time model fits the multi-stage serving model with endogenous memory growth. We added real-GPU validations in the Additional Experiments appendix, including direct A100 timing measurements and end-to-end GPU runs implemented with SGLang 0.5.7 (SGLang Team, 2024). The main policy comparisons remain Vidur simulations configured for A100 hardware; the real-GPU experiments provide two forms of support: direct timing calibration for the simulator and end-to-end implementation evidence for WAIT's scheduling logic. We also explain why the model is particularly well matched to prefill-decode disaggregated systems such as Splitwise and DistServe (Patel et al., 2024; Zhong et al., 2024), where separating prefill from decode makes aggregate KV-cache usage a natural state variable for iteration time. For richer calibrated service-time curves, the formal guarantees would require corresponding model-specific analysis, but the same scheduling perspective remains useful: admission and batching decisions must control the number of resident prompts and their composition across prefill and decode stages because these quantities drive both memory pressure and future iteration times.
- **Clarified the theoretical contribution.** We removed the misleading "heavy traffic" terminology from the narrative and now describe the guarantees as asymptotic guarantees under fixed load. The main text

now explains why the problem differs from classical formulations with exogenous service requirements: KV-cache memory grows endogenously with decode progress, evictions can restart partially completed work, and unknown output lengths are revealed only through the service trajectory. The WAIT and Nested WAIT thresholds reduce this high-dimensional state-action space to tractable boundary queues.

- **Expanded and reorganized the experiments.** Section 6 now reports latency and effective-completion-rate curves over arrival-rate grids that include underloaded, near-capacity, and overloaded regimes. The Additional Experiments appendix adds simulator-to-GPU validation on an A100, supplemental end-to-end real-GPU experiments, repeated-run variability near the transition from near-overloaded to overloaded, threshold-without-waiting comparisons, and a prefill-decode-disaggregated deployment study. For the real-data experiment, we compare the latency of different policies and use the effective-completion-rate curve to show where each policy continues to track the offered load.
- **Updated positioning and scope.** The Introduction and related-work discussion now clarify that our contribution is a modeling and scheduling contribution for multi-stage LLM inference with endogenous memory growth, not a claim of a new heavy-traffic limit. We also added discussion of concurrent theoretical work and prefill-decode disaggregated systems, where the decode-only setting makes the linear iteration-time model especially natural.
- **Improved exposition and organization.** We rewrote large portions of Sections 1–6 and the appendices for clarity, removed ambiguous terminology, added mechanism-level explanations after algorithms and theorems, and modularized the appendix proofs.

Main concern	**How the revised manuscript addresses it**
Throughput with exogenous arrivals	Recasts the objective around stability-region expansion, effective throughput net of evictions, latency, and TTFT.
Memory realism	Counts retained KV caches of admitted requests, explains eviction-induced restarts, and studies near-capacity and long-decode regimes.
Linear time model	States the KV-cache-driven scope, reports A100 validation results, and discusses PD-disaggregated deployments.
Theoretical novelty	Positions WAIT/Nested WAIT as fluid-guided threshold policies that reduce a high-dimensional memory-growth control problem to tractable boundary queues.
Writing quality	Rewrites Sections 1–6, adds notation summary and examples, and reorganizes the appendices around theorem-specific proof logic.

Response to the Area Editor

Comment. *The paper requires a complete expositional overhaul; the original presentation had notation inconsistencies, insufficient intuition, and proof summaries that were too high level to be useful.*

Our response. We are grateful for the editor's detailed guidance on exposition and presentation. It helped us identify the main presentation problem: the earlier version moved too quickly from the serving system to the

mathematical abstraction. In the revised manuscript, we therefore significantly polished and reorganized the exposition so that the reader first sees the operational mechanism, then the mathematical abstraction, and only then the policy and analysis. Section 2 now introduces notation as it enters the model, with a consolidated notation table moved to the Notation Summary appendix. Sections 4 and 5 now include examples and mechanism-level explanations immediately before the algorithms, and the theorem discussions explain what the memory conditions mean in terms of batching, KV-cache growth, and boundary queues. The appendices have also been modularized so that each theorem proof has its own proof section and the main text explains the proof logic before deferring details.

Changes in the revised manuscript.

- Rewrote Section 2 around prompts, inference stages, batching, memory, and the objective.
- Added a notation summary in the Notation Summary appendix.
- Added examples and explanatory figures for WAIT and Nested WAIT in Sections 4 and 5.
- Reorganized the proof appendix into modular sections for Propositions and Theorems 1–3.
- Added theorem-interpretation paragraphs that connect each guarantee to the primitive queueing and memory mechanisms.

Comment. *The model assumptions need either modification or better justification, especially the linear iteration-time model and the treatment of memory, OOM, and preemption.*

Our response. The comments on the model assumptions were especially helpful because the memory mechanism is central to the paper. In the revision, we retained the tractable model but made its scope explicit. The linear iteration-time model separates the fixed per-iteration overhead from the marginal cost of serving active KV caches. We note that heterogeneous-prefill regimes may require richer calibrated service-time curves, whereas prefill-decode disaggregated deployments make the approximation especially appropriate (Patel et al., 2024; Zhong et al., 2024). The revised text also clarifies how the modeling insight extends beyond the linear specification: even with a richer service-time curve, the scheduler still has to regulate the batch composition across prefill and decode stages, because this composition determines memory pressure and future batch service times. To avoid relying only on simulation, the Additional Experiments appendix compares Vidur predictions (Agrawal et al., 2024) with direct A100 measurements (NVIDIA, 2026) on a single-type calibration grid, and separately reports end-to-end experiments using SGLang (SGLang Team, 2024). The memory model has also been clarified: admitted requests that remain active keep their KV caches on the GPU and count against the memory constraint. If memory is exceeded, the server evicts and restarts an in-progress request, which is counted as lost useful work in the effective-throughput metric.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the linear iteration-time model and its KV-cache-driven scope.
- Revised Section 2.3 to state that the memory constraint applies to all in-system KV caches retained on the GPU.
- Added real-GPU validation in the Additional Experiments appendix, with A100 measurements and a fitted iteration-time model.

- Added a prefill-decode-disaggregated experiment in the Additional Experiments appendix showing that the same threshold logic applies in a decode-only serving scenario.
- Revised Section 6 and the long-decode eviction table to report eviction-induced restart rates where relevant.

Comment. *The theoretical novelty should be clarified. If the contribution is modeling and problem structuring rather than a new stochastic-process theorem, that should be stated clearly.*

Our response. This suggestion led us to sharpen the paper's positioning. Rather than presenting the contribution as a heavy-traffic theorem for a standard open queue, the revised manuscript emphasizes the modeling and algorithm-design contribution: LLM inference creates a high-dimensional control problem because memory consumption grows with decode progress, output lengths may be unknown, and eviction can erase partially completed work. WAIT and Nested WAIT impose threshold structure that keeps the system close to the fluid equilibrium and turns the analysis into tractable boundary processes. The mathematics then formalizes how this structured policy controls memory growth under the stated memory conditions and tracks the fluid effective-throughput benchmark in the asymptotic regime.

Changes in the revised manuscript.

- Reframed Section 3 as a fluid model and equilibrium analysis rather than a throughput-maximization problem at a fixed arrival rate.
- Rewrote Sections 4.1–4.2 to explain why threshold-based admission reduces the state-action space.
- Rewrote Section 5 to explain how Nested WAIT handles unknown output lengths through segment boundaries and binomial thinning.
- Removed misleading "heavy traffic" language from the main narrative and replaced it with asymptotic language.

Comment. *The paper should better connect the LLM-inference domain to existing stochastic-modeling and scheduling literature.*

Our response. We are grateful for the editor's suggestion to connect the LLM-inference domain more directly to existing stochastic-modeling and scheduling literature. Following this guidance, the revised Introduction now discusses concurrent theoretical work on online scheduling, competitive ratios, regret, robust prediction, and stochastic batch processing for LLM inference. We also sharpened the distinction between our queueing-control setting and classical formulations with exogenous service requirements. In our model, admitted prompts move through prefill and decode stages, their KV-cache requirements grow over time, and the system may restart work due to endogenous memory growth. The scheduler therefore controls not only which jobs enter service, but also the composition of the GPU-resident workload across prefill and decode stages. This queueing-control problem created by endogenous memory growth motivates the fluid-guided threshold structure developed in the paper. We also discuss prefill-decode disaggregated systems (Patel et al., 2024; Zhong et al., 2024), where the decode-side workload aligns closely with the linear iteration-time model.

Changes in the revised manuscript.

- Expanded the Other Related Work subsection in the Introduction.
- Added discussion of concurrent LLM-inference scheduling theory and the distinction between their timing/information models and our memory-dependent batch model.
- Added connections to Splitwise and DistServe-style prefill-decode disaggregation.

Response to Reviewer 1

We thank Reviewer 1 for the careful reading and for emphasizing the importance of modeling clarity, notation consistency, and memory-overflow safeguards. These comments shaped the rewrite of Section 2 and the memory discussion throughout the paper.

R1.1. Memory overflow and KV-cache capacity

Comment. *The reviewer asks how the proposed algorithms ensure that KV-cache usage does not exceed capacity, especially under heavy workloads and uncertain output lengths.*

Our response. This is an important practical question, and we appreciate the reviewer asking us to address a point that is central to the paper. The original manuscript did not make the memory accounting explicit enough. In the revised manuscript, the relevant memory is the total KV cache retained by all admitted in-system prompts on the GPU. If the system admits too much work, decode progress increases KV-cache usage and may force the serving system to evict and restart a request. The proposed policies control this risk by regulating how many prompts enter and advance through decode. For known output lengths, WAIT uses type-specific thresholds derived from the fluid equilibrium and prevents eviction under the stated memory condition. For unknown output lengths, Nested WAIT adds segment-level thresholds and moderate safety buffers so that prompts that survive to later decode segments do not exceed the memory budget with high probability.

Changes in the revised manuscript.

- Added the memory-growth example and memory-constraint discussion in Section 2.3.
- Added Example 2 in Section 4 to show how FCFS can trigger eviction cascades even when the nominal capacity is sufficient.
- Added Example 3 and the Nested WAIT explanation in Section 5 to show how unknown output lengths are handled through segment boundaries.
- Added a long-decode near-capacity eviction diagnostic in Section 6, where Sarathi incurs a positive eviction-induced restart rate while WAIT avoids eviction.

R1.2. Linearity of Equation (1)

Comment. *The reviewer asks for additional justification of the linear iteration-time assumption and for discussion of possible nonlinear relationships across GPU architectures.*

Our response. We expanded the discussion of Equation (1) and clarified its intended operating regime. The linear form is a parsimonious approximation when prompts spend most iterations in decode and the marginal cost is well captured by total active KV-cache length. We do not claim that this model captures every possible hardware and workload regime. Mixed prefill-decode iterations, especially with long or highly heterogeneous prefill lengths, may require additional prefill-attention or hardware-specific terms. The revised manuscript states this limitation and adds two forms of empirical support: Section 2 reports direct A100 profiling of batch inference time against total cached tokens (NVIDIA, 2026), and the Additional Experiments appendix compares Vidur's iteration-time predictions (Agrawal et al., 2024) with direct A100 measurements. It also notes that, under richer calibrated service-time models, the same admission-control viewpoint continues to organize the problem: the scheduler must decide how many prompts to keep resident and how they are distributed across prefill and decode stages, since those choices shape both memory growth and subsequent iteration times.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the fixed-overhead and marginal-KV-cache components of Equation (1).
- Added A100 batch-inference-time profiling in Section 2 and Additional Experiments appendix validation comparing Vidur predictions with direct A100 measurements. The appendix now distinguishes the Llama-2-7B profiling-supported batch-size range $B \leq 128$ and reports a separate test of extrapolation to batch size $B = 256$.
- Added discussion in the Introduction and Additional Experiments appendix that prefill-decode disaggregation is a natural setting where the decode-side linear approximation is especially accurate.
- Added discussion that richer calibrated service-time curves can be incorporated conceptually, while the formal results in this paper focus on the linear multi-stage model with endogenous memory growth.

R1.3. Notation inconsistencies

Comment. *The reviewer notes that the original manuscript used similar notation for decode stage, prompt length, output length, and current processing stage.*

Our response. The reviewer is right that these notation conflicts created avoidable friction in the original manuscript. The revised manuscript separates these concepts. Decode stage is denoted by the stage index s. Prompt length, output length, and type-dependent lengths use separate notation. To reduce the burden on the reader, we moved the full notation table to the Notation Summary appendix and introduce symbols narratively as they first appear in the main text.

Changes in the revised manuscript.

- Revised Section 2 to introduce stage notation consistently.
- Added the Notation Summary appendix table collecting the main symbols used throughout the paper.
- Updated the algorithms and theorem statements to use the same stage and threshold notation.

R1.4. Policy space and batching operations

Comment. *The reviewer asks for a clearer mathematical characterization of the policy space* Π *and the batching operations implied by a policy.*

Our response. To make the policy class concrete, we added a dedicated objective and policy-space subsection. The revised text explains that a policy observes the current queues and KV-cache state, chooses which prompts to admit or advance, and must respect the information available at the decision epoch. This makes the algorithms easier to interpret as concrete elements of the policy class rather than standalone procedures introduced after the model.

Changes in the revised manuscript.

- Revised Section 2.4 to define the objective, the policy class, and the main performance metrics.
- Clarified the relation between policy decisions, batch formation, memory consumption, latency, TTFT, and effective throughput.

R1.5. Need for intuition after definitions and algorithms

Comment. *The reviewer asks for more explanatory text after key definitions and Algorithm 1.*

Our response. We took this suggestion as a guide for the main-text rewrite. Section 2 now explains each modeling component through the running LLM-inference example. Section 4 begins with a concrete FCFS failure example before introducing WAIT. Section 5 begins with an unknown-output-length example before introducing Nested WAIT. The algorithm descriptions now explain why thresholds regulate both admission and the downstream composition across prefill and decode stages.

Changes in the revised manuscript.

- Added explanatory text around the introductory inference figure and the batching example figure.
- Added Example 2 before WAIT and Example 3 before Nested WAIT.
- Added mechanism paragraphs after Algorithm 1 and Algorithm 2.

R1.6. Typos

Comment. *The reviewer notes several typos, including d versus* d_1 *and "Thoughput" versus "Throughput."*

Our response. We have corrected these issues and appreciate the reviewer noting them. We also conducted a full consistency check for notation, terminology, cross-references, and repetitive phrasing.

Changes in the revised manuscript.

- Corrected the throughput macro.
- Corrected d, d_0, and d_1 inconsistencies in the model and fluid sections.
- Removed duplicate or inconsistent notation flagged in the reviewer comments.

Response to Reviewer 2

We sincerely thank Reviewer 2 for the detailed and constructive technical report. The comments on the time model, the role of the memory assumption, the proof structure, and experimental reproducibility led to several substantive improvements in both the main text and the appendix.

R2.1. Scope of the linear iteration-time model

Comment. *The reviewer suggests that Equation (1) may omit prefill attention, linear layers, and decode attention terms, and asks which asymptotic regime the paper targets.*

Our response. This concern correctly identifies the main scope issue behind Equation (1). The revised manuscript now states that Equation (1) targets regimes in which marginal iteration time is well captured by total active KV-cache length. Mixed prefill-decode iterations, especially with long or highly heterogeneous prefill lengths, may require additional prefill-attention or hardware-specific terms. Rather than claiming universality, we now position Equation (1) as a transparent approximation that captures the admission-control trade-off central to the paper: admitting more prompts increases batching efficiency but also raises iteration time and memory consumption through larger KV caches. The revised text further explains that the broader scheduling idea does not depend on treating the linear model as universal. With a different calibrated service-time curve, one would redo the analysis for that curve, but the same state variables remain operationally central: resident prompt counts, composition across prefill and decode stages, and KV-cache growth. We also added A100 validation (NVIDIA, 2026) and explain why prefill-decode disaggregated deployments (Patel et al., 2024; Zhong et al., 2024) provide a natural setting for this approximation.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the operating regime of Equation (1).
- Replaced the earlier L20 profiling plot with an A100 profiling figure using the same GPU model as the main Vidur configuration. The revised figure reports A100 80GB measurements for Llama-2-7B (NVIDIA, 2026; Hugging Face, 2025) with input length 256, output length 20, and the number of requests in the batch varying through the profiled range, plus a separate extrapolation test to $B = 256$. Each point corresponds to a batch state in which active requests may be at different decode stages, so the

x-axis reports the total number of cached tokens across active requests and the y-axis reports measured batch inference time.

- Added the simulator-calibration figure in the Additional Experiments appendix, comparing Vidur iteration-time predictions with direct A100 measurements.
- Added discussion of prefill-decode disaggregation, where the decode server removes the prefill-attention terms and the linear KV-cache approximation is especially appropriate.
- Added scope language explaining that more general calibrated service-time curves can be studied through the same admission-and-batching perspective, although the formal guarantees in the manuscript are stated for the linear model.

R2.2. Clarifications in Section 2

Comment. *The reviewer provides several minor comments on notation, prefill throughput, the policy class, and whether preemption is permitted.*

Our response. These detailed comments were helpful because they pointed to several places where Section 2 imposed unnecessary burden on the reader. In the revision, the stage notation has been cleaned up, the role of prefill and decode is described explicitly, and the objective discussion now clarifies the metrics studied in the paper. The revised model does not rely on costless swapping of inactive KV caches. Requests that are admitted and remain active retain their KV caches on the GPU and count toward memory consumption; if memory is exceeded, the serving system evicts and restarts work, which is treated as lost useful service.

Changes in the revised manuscript.

- Rewrote Section 2.1–2.4 for notation and policy clarity.
- Added the Notation Summary appendix.
- Clarified that the memory constraint applies to retained KV caches of in-system requests.

R2.3. The assumption $C \geq M^*$

Comment. *The reviewer notes that $C \geq M^*$ may be a nontrivial requirement, especially for very long outputs, and asks whether the experiments satisfy or violate this condition.*

Our response. The point about $C \geq M^*$ is important, especially for long outputs. We now treat M^* as the memory requirement for the fluid equilibrium, not as an additional capacity assumption. The condition $M^* \leq C$ says that the workload is stabilizable in the fluid model. The policy question is whether an online scheduler can realize this stability in the stochastic system while memory grows over time. To connect this condition to observed system behavior, the revised experiments now examine underloaded, near-overloaded, and overloaded regimes across synthetic and heterogeneous workloads. We also added a real-data memory check: for the real dataset studied in Section 6, Section 6 reports representative values of $M^*(\lambda)$ computed from the empirical prefill/decode length distribution and compares them with the memory capacity C. Across

these regimes, the empirical patterns are consistent with the theory's mechanism: threshold-based admission helps the system exploit available memory and server capacity while reducing eviction-induced restarts. As memory pressure increases, WAIT and Nested WAIT keep completed work closer to usable capacity by regulating when prompts enter and advance through decode, rather than relying on excess memory.

Changes in the revised manuscript.

- Revised the discussion around M^* in Sections 3 and 4.
- Added a real-data $M^*(\lambda)$ versus C table in Section 6 for the real dataset.
- Added long-decode experiments in Section 6, including `p512d1000`, to test near-overloaded and overloaded behavior under severe memory pressure.
- Added the prefill-decode-disaggregated deployment subsection in the Additional Experiments appendix, where decode-side memory pressure is isolated.

R2.3a. Segment-wise design for unknown or long output lengths

Comment. *The reviewer notes that maintaining a separate stage for every possible output length may be impractical in long-output settings.*

Our response. The distinction between the analytical construction and implementable segment design is important. The revised paper separates the clean theorem statement from the implementation-oriented design. The main theorem presents the analytically transparent case in which output lengths map to segment boundaries. The Extensions appendix then discusses general segment designs that group nearby output lengths into fewer segments. This is the form used in the real-data experiments: segment-level thresholds preserve the same continuation-probability logic while avoiding the need to maintain a separate threshold for every individual decode length.

Changes in the revised manuscript.

- Revised Section 5 to explain segment boundaries as the points where output-length information is revealed.
- Added Extensions appendix discussion of general segment design and coarser segmentations.
- Revised Section 6.2 to describe how empirical continuation probabilities determine the segment thresholds in the real-data workload.

R2.4. Proof of Theorem 1 and decode-stage dynamics

Comment. *The reviewer asks how the proof accounts for l' decode stages and whether completions occur only after enough batch executions.*

Our response. The proof now gives a more explicit account of the decode-stage dynamics. WAIT is designed so that once a type is scheduled, threshold-sized groups advance together through the decode stages. This

threshold structure is what permits the proof to reduce the multi-stage process to a boundary queue for each type. The main text now explains this dimensionality reduction before the theorem discussion, and the appendix proof explicitly separates the batch index from the decode-stage index.

Changes in the revised manuscript.

- Revised Section 4.2 to explain how threshold-sized batches move through decode stages.
- Revised the proof appendix for Theorem 1 to define the batch index separately from the stage index.
- Added explanatory text connecting queue accumulation to throughput loss in the proof of Theorem 1.

R2.5. Thresholds without waiting

Comment. *The reviewer asks what happens if the algorithm uses thresholds but does not wait for enough prompts to accumulate.*

Our response. To isolate the role of waiting, we added a targeted comparison in the Additional Experiments appendix. The threshold-only variant imposes an upper bound on in-system requests but does not wait to form threshold-sized batches. The comparison shows that the waiting component has a regime-dependent role. At low arrival rates, waiting can add modest delay because the server has sufficient capacity to process requests immediately. Near the overload boundary, waiting becomes useful because it regulates when work enters service and improves batch formation. Once the system is overloaded, the aggregate upper bound dominates and the two variants behave more similarly.

Changes in the revised manuscript.

- Added the "Threshold without waiting" subsection in the Additional Experiments appendix.
- Added the wait-versus-threshold-only figure reporting the latency comparison across arrival rates.

R2.6. Role of the asymptotic assumption

Comment. *The reviewer asks what role the original heavy-traffic assumption plays and whether the theorem can be stated without it.*

Our response. We appreciate the reviewer making this distinction explicit. The analysis does not take a classical heavy-traffic limit in which the load approaches the boundary of the stability region. It studies asymptotic averages under fixed load and a scaled observation horizon. We have removed the heavy-traffic framing from the narrative and now describe the result as an asymptotic guarantee. The assumption is used to compare the stochastic system against the fluid benchmark over a scaled horizon, not to claim a conventional heavy-traffic diffusion limit.

Changes in the revised manuscript.

- Replaced "heavy traffic" terminology in the main text with asymptotic terminology.
- Revised the theorem discussion in Sections 4 and 5 to state the role of the limit more directly.

R2.7. Formula and proof-detail comments

Comment. *The reviewer lists several formula checks and appendix proof clarifications, including d_0 versus d_1, M^π versus M^*, undefined notation, and an alternative proof idea.*

Our response. We treated these formula and proof comments as a guide for revising the appendix. The revised manuscript corrects the d_0/d_1 inconsistencies, clarifies the distinction between M^* and M^π, defines segment probabilities and safety-buffer terms before the Nested WAIT theorem, and reorganizes the proof appendix around the main theorem statements. We also adopted the spirit of the reviewer's alternative proof suggestion by making the coupling argument more direct and by separating the dominating process from the original queue process.

Changes in the revised manuscript.

- Corrected the d_0/d_1 inconsistencies in Section 3.
- Revised Section 5.2 to define p_k, n_k, M^π, and the safety-buffer interpretation before Theorem 2.
- Revised the proof appendix for theorem-specific proof organization.
- Added citations and references to standard queueing tools where appropriate.

Detailed correction checklist. We also checked the detailed notation and proof points in the report. In particular, the revision (i) separates prompt type $j(i)$, prefill/decode stage s_i^t, prefill length l_j, and decode length l'_j; (ii) states that $s_i^t = 0$ denotes prefill; (iii) corrects the d_0/d_1 inconsistencies in the fluid calculations; (iv) defines $\Delta T(\cdot)$, M^*, and M^π before they are used in the theorem statements; (v) states the advancement logic through prefill and decode used in Theorem 1; (vi) distinguishes the embedded review intervals used in the WAIT proof from actual event-driven batches; (vii) adds the resident-memory and post-review boundary-queue conventions in the WAIT and Nested WAIT proofs; (viii) cites standard reflected-random-walk and Kingman-type tools rather than reproving those facts from first principles; and (ix) removes or corrects the undefined or ambiguous symbols flagged in the appendix.

R2.7a. Appendix organization and use of standard stochastic results

Comment. *The reviewer and the Area Editor suggest that the appendix should be easier to verify and should connect more directly to standard stochastic-process tools.*

Our response. In response, we reorganized the proof appendix so that its structure is easier to verify. It now separates proposition proofs, the WAIT theorem proof, the Nested WAIT theorem proof, the time-varying extension, and supporting lemmas. Within the theorem proofs, we use standard stochastic-process tools from queueing analysis explicitly: a Lindley-type recursion for the threshold queue, random-walk maximum bounds for queue accumulation, and martingale bounds for the Nested WAIT safety buffer. The main text now explains why the policy creates the reduced process before the appendix analyzes it.

Changes in the revised manuscript.

- Split the proof appendix into theorem-specific proof sections.

- Added definitions of the batch index, stage index, and coupled process before using them in proofs.
- Added citations to standard queueing tools where they replace or streamline first-principles derivations.

R2.8. Experimental parameters and reproducibility

Comment. *The reviewer asks for batch-size, memory, capacity, Sarathi configuration, and Nested WAIT parameter details in Section 6.*

Our response. We substantially rewrote Section 6 and the Additional Experiments appendix so that the main settings are explicit rather than implicit. The main text now states the hardware, simulator, baseline configurations, WAIT/Nested WAIT threshold construction, arrival-rate grids used in the experiments, and number of replications. The baselines are described as greedy FCFS-based policies; Sarathi differs by using chunked prefill (Agrawal et al., 2023), and the memory-reservation rule used by both baselines is stated explicitly. We also added a memory-cap calculation for the A100 80 GB / Llama-2-7B configuration (NVIDIA, 2026; Hugging Face, 2025): under the memory accounting used in our experiments, after the model weights and the baseline memory margin, the baseline-memory runs use a KV-cache cap of approximately 1.37×10^5 tokens. The token calculation follows the standard key/value-cache accounting for Transformer decoding (Atharva, 2026). The same paragraph now states that if the resident KV cache exceeds this cap, the server evicts and restarts in-progress requests. For the synthetic single-type workload, WAIT calibrates its threshold scale on the finite grid $\mathrm{tl} \in \{20, 30, \ldots, 200\}$ and converts the selected cap into per-stage limits. For Nested WAIT, the revised text explains the distribution-aware calibration rule: the system-wide batch-size cap and segment count are selected on finite grids using the workload distribution and offered arrival rate, and are then converted into segment and per-stage thresholds through continuation probabilities. The text also clarifies that tl is a threshold scale, not a claim that tl longest-context requests can reside on the GPU simultaneously; the actual resident KV-cache usage is governed by the memory cap. For the real-data experiment, the manuscript reports the $\mathrm{tl} \in \{20, 40, \ldots, 200\}$ grid, segment-count grid $L \in \{1, 2, 3, 4, 5, 10, 20\}$, and margin parameter $\eta = 0.05$ used in this calibration. We present this as a rate-aware calibration of a distribution-aware policy: the workload distribution and offered arrival rate set the parameters before online scheduling, while each scheduling decision does not use the realized final output length of the request being scheduled. The GPU appendix separately reports rate-specific parameter settings for the real-data GPU runs.

Changes in the revised manuscript.

- Added hardware and baseline-configuration paragraphs at the start of Section 6.
- Added a WAIT/Nested WAIT settings paragraph in Section 6.
- Revised the real-data subsection to explain the dataset, sampling, segment construction, and threshold allocation.
- Added real-GPU implementation details, including a parameter table for the real-data GPU run.

R2.9. Simulation fidelity and real-GPU validation

Comment. *The reviewer recommends validating Vidur against actual GPU measurements.*

Our response. We added both forms of validation requested by the reviewer. The Additional Experiments appendix compares Vidur predictions with direct A100 measurements for Llama-2-7B on a single-type calibration grid. The raw Vidur profiling data for this Llama-2-7B/A100 setting cover batch sizes through $B = 128$; we report the profiled range and separately test extrapolation to batch size $B = 256$ against a direct A100 measurement. The fitted validation curve has small mean absolute percentage error and high R^2, and the extrapolation to $B = 256$ remains close to the measured A100 runtime. We also added end-to-end real-GPU experiments using SGLang 0.5.7 (SGLang Team, 2024), comparing WAIT with the baseline scheduler for a single-type workload and Nested WAIT for the real-data workload distribution. The main policy comparisons in Section 6 remain Vidur simulations configured for A100 hardware, while these real-GPU experiments provide direct timing calibration for the simulator and implementation evidence for the WAIT/Nested WAIT scheduling logic.

Changes in the revised manuscript.

- Added the simulator-calibration figure in the real-GPU validation appendix.
- Added end-to-end real-GPU comparisons on single-type and real-data workloads.
- Added references to the validation from Section 6.

R2.10. Long outputs

Comment. *The reviewer asks how the algorithm behaves when output length exceeds 1000 tokens.*

Our response. This is a natural stress test for the proposed memory-control mechanism, and we added a long-decode workload to Section 6. The workload `p512d1000` keeps the prefill length fixed and increases the decode length to 1000. This setting stresses KV-cache growth and causes all policies to lose stability at much lower arrival rates. WAIT has the lowest latency near the boundary and avoids eviction-induced restarts in the near-capacity diagnostic, illustrating that threshold-based admission remains useful when decode-side memory growth is pronounced.

Changes in the revised manuscript.

- Added the long-decode regime in Section 6.1.
- Added the long-decode figure and near-capacity eviction table for latency, effective completion rate, and eviction-induced restart evidence.

Response to Queueing, Stability, and Latency Concerns

We also thank the referees and editors for the detailed critique of the queueing interpretation, latency evidence, and theoretical positioning. This critique identified several important weaknesses in the original

submission: the throughput interpretation under exogenous arrivals, the misuse of heavy-traffic terminology, the need for meaningful latency experiments, and the need to explain why the theory is not merely a routine positive-recurrence argument. Addressing these points led to the largest changes in the revision.

D.1. Heavy-traffic terminology

Comment. *The reviewer notes that the original "heavy traffic" analysis was not a conventional heavy-traffic limit; it was a long-horizon limit under fixed load.*

Our response. We appreciate the referees identifying this terminology issue. The revised paper does not present the results as a classical heavy-traffic analysis. Instead, it explains that the theoretical results compare the stochastic system to the fluid benchmark in an asymptotic regime under fixed arrival rates. The revision also clarifies the role of the theory: the contribution is to structure the LLM-inference control problem through fluid-guided thresholds and to prove that the resulting policy tracks the benchmark, not to derive a new diffusion limit.

Changes in the revised manuscript.

- Replaced heavy-traffic terminology in the main narrative.
- Revised Sections 4 and 5 theorem discussions to state the asymptotic regime more accurately.

D.2. Throughput with exogenous arrivals

Comment. *The reviewer points out that, with exogenous arrivals, a stable system completes work at the arrival rate, so the relevant question is the stability region rather than throughput at a fixed arrival rate.*

Our response. This clarification led us to reframe the objective around stability and useful completed service. The revised Section 2 objective discussion now explains that a stable system cannot complete more useful work than the offered load. The operational challenge is whether a policy can convert the offered work into completed service while avoiding idle capacity, memory overflow, and eviction-induced restarts. Section 3 uses the fluid model to identify the memory requirement and composition across prefill and decode stages required for stability under the idealized model. Section 6 then reports latency and effective-completion-rate curves over the arrival rates in each experiment, so departures from stable operation appear through latency growth and completion-rate saturation.

Changes in the revised manuscript.

- Revised Section 2.4 to define effective throughput and latency under a fixed external arrival process.
- Revised Section 3 to explain how the fluid model identifies the stability region and the memory required to sustain it.
- Revised Section 6 to explain why stable policies have similar completion rates below the boundary and why latency curves are needed to compare policies within the stable region.

We also changed the terminology around M^*. The revised manuscript no longer describes M^* as a "throughput-optimal memory" level. It is the memory requirement needed to sustain the fluid equilibrium for the arrival vector under consideration. For a fixed physical capacity C, the condition $M^* \leq C$ determines whether that load lies inside the fluid stability region. This distinction matters under exogenous arrivals: increasing memory does not raise completed work above the offered load in a stable system, but insufficient memory or poor admission control can shrink the stability region and reduce effective completed service through evictions.

D.3. Positive recurrence and theoretical significance

Comment. *The reviewer argues that bounded latency under a stable policy may follow from standard positive-recurrence arguments unless the paper explains the nontrivial part of the analysis.*

Our response. This point is central to the theoretical positioning, and we appreciate the referees asking us to make it explicit. We have revised the theory sections to distinguish the standard queueing implication from the nonstandard part of our problem. The difficult step is not that a stable queue has finite delays; it is designing a policy whose high-dimensional LLM-serving state can be controlled under growing memory consumption and eviction risk. The revised fluid section now makes the source of this difficulty explicit: for a single type, away from the critical denominator, the memory required to sustain the fluid equilibrium grows quadratically in the decode length, because both the number of active prefill/decode stages and the average KV-cache size across the batch composition increase. Thus even a small amount of long-decode traffic can dominate the memory condition defining the fluid stability region. The WAIT proof then connects the actual event-driven policy to an auxiliary embedded full-threshold process observed at deterministic review epochs. The sample path coupling is the nonstandard step: it shows that controlling the threshold profile also controls the batch composition across prefill and decode stages, and that the embedded process tracks both entry starts and downstream stage advancement up to a negligible one-interval lag. Nested WAIT further handles unknown output lengths by using segment boundaries: prompts reveal their output length only when they complete at a boundary or survive to the next segment. The boundary-queue coupling exploits this structure to control memory overflow without tracking every prompt across every prefill/decode stage.

Changes in the revised manuscript.

- Rewrote Section 4.2 to explain the dimensionality reduction created by WAIT.
- Rewrote Section 5.2 to explain the boundary-queue coupling created by Nested WAIT.
- Added discussion in Section 3 showing why long-decode requests can dominate the memory requirement for the fluid stability region.
- Revised the theorem interpretation paragraphs to connect memory conditions to overflow prevention rather than simply stating bounded latency.

This point guided the revision: the policy is not merely a work-conserving batching rule, but a dimensionality-reduction device. Without a well-structured threshold policy, the primitive state includes the number of prompts at each decode stage, their retained KV caches, type uncertainty, and the possibility of eviction-

induced restarts. The threshold design is what creates the lower-dimensional queueing object analyzed in the proof.

D.4. Latency experiments and unstable regimes

Comment. *The reviewer notes that latency comparisons in unstable regimes can be misleading and recommends plotting mean latency as a function of arrival rate.*

Our response. This suggestion led us to rework the numerical section around arrival-rate curves. The main figures now plot mean end-to-end latency versus arrival rate and pair it with effective completion rate. Each latency point is computed over a central measurement window after the initial transient, using the same window across policies and replications. This presentation shows the underloaded, near-capacity, and overloaded regimes in the same figure. At rates where a policy tracks the offered load, latency is the informative comparison. Around the transition from near-overloaded to overloaded, latency growth signals that the policy is approaching the range it can sustain over the simulated horizon. Above that range, effective completion rate and eviction-induced restarts show how much useful work each policy can still complete.

Changes in the revised manuscript.

- Added arrival-rate experiments for single-type, two-type, long-decode, and real-data workloads in Section 6.
- Added a long-horizon simulation check near the two-type transition from near-overloaded to overloaded.
- Reported each plotted arrival-rate point as an average over repeated simulation replications.
- Added repeated-run variability analysis in the Additional Experiments appendix.

The revised numerical section now separates low-load behavior from the near-overloaded and overloaded regimes. At low load, the policies are often close. In the near-overloaded and overloaded regimes, WAIT/Nested WAIT performs better because the threshold controls keep the in-system population closer to balance and reduce eviction-induced waste. For both the synthetic and real-data experiments, we report mean latency together with effective completion rate. The effective completion rate tracks the offered arrival rate in the stable region and falls below it once the policy can no longer sustain the load.

D.5. Algorithmic contribution and latency

Comment. *The reviewer characterizes WAIT as a pipeline-filling idea and asks for a stronger explanation of latency effects and algorithmic contribution.*

Our response. The pipeline interpretation is a useful starting point, and we appreciate the reviewer raising it because it clarified what the manuscript needed to explain beyond pipeline filling. The revised paper emphasizes that KV-cache memory grows with decode progress, admitted jobs can later cause overflow, and evictions restart useful work. WAIT is not only filling a pipeline; it regulates the population across prefill and decode stages so that arrivals, completions, and memory growth remain balanced. Nested WAIT extends

the same principle when output length is unknown, delaying type separation until the prompt's own decode trajectory reveals whether it is short or long. The numerical section now reports latency across arrival rates to show where these controls matter.

Changes in the revised manuscript.

- Added and revised examples in Sections 4 and 5 to explain WAIT and Nested WAIT through concrete system trajectories.
- Revised Section 6 to emphasize latency differences near capacity and the larger arrival-rate range sustained by the proposed policies.
- Added threshold-without-waiting and real-GPU comparisons in the Additional Experiments appendix.

D.6. Memory constraint and preempted jobs

Comment. *The reviewer asks whether the memory constraint applies only to jobs in the current batch and whether preserving preempted KV caches elsewhere is realistic.*

Our response. The original memory description left room for exactly this ambiguity. We revised the model to remove it. The memory constraint applies to the KV caches of admitted in-system requests retained on the GPU. Requests that are not selected in the current iteration remain in their current queues, and those already in decode retain their KV caches. We do not rely on moving inactive KV caches to CPU memory or SSD at zero cost. If the retained GPU KV-cache memory exceeds capacity, the serving system must evict and restart an in-progress request. This loss channel motivates the admission-control problem studied in the paper.

Changes in the revised manuscript.

- Revised Section 2.3 to state the memory accounting explicitly.
- Revised Algorithm 1 and Algorithm 2 explanations to state that unselected decode prompts retain their KV caches.
- Added eviction-induced restart diagnostics in Section 6.

D.7. Writing quality, definitions, and interpretation

Comment. *The reviewer notes that the original manuscript was unclear, with incomplete definitions, little interpretation of formulas, and notation inconsistencies.*

Our response. This assessment became a central goal of the revision. The original manuscript asked the reader to infer too much from formulas and algorithms. The revision contains a substantial rewrite of the main text and appendices. The fluid model is now developed more slowly, with each formula followed by its operational interpretation. The algorithms are introduced through examples before pseudocode. The theorem statements are followed by paragraphs explaining what the memory terms mean and how the proof reduces the original system to tractable queues.

Changes in the revised manuscript.

- Rewrote Sections 2–5 for notation, flow, and interpretation.
- Added the notation table in the Notation Summary appendix.
- Added explanatory paragraphs after the fluid-model formulas, algorithms, and theorems.
- Reorganized the appendix proofs and removed several sources of notational ambiguity.

Closing

We again thank the Area Editor, the Associate Editor, and the referees for the detailed guidance. The revised manuscript is substantially different from the original submission: the problem framing is now centered on stability-region expansion, eviction prevention, and latency; the model assumptions are stated more transparently; the theory is positioned as a fluid-guided policy design and dimensionality-reduction argument; and the experiments now address the main queueing diagnostics requested in the reports through arrival-rate curves, effective-completion-rate plots, long-horizon checks, and supplemental physical-GPU timing and implementation checks. We hope that the revised version makes the contribution and limitations of the work much clearer.

References

Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani B, Ramjee R, Tumanov A (2024) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6:351–366.

Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* .

Atharva (2026) The kv cache: How it eliminates redundancy. https://huggingface.co/blog/atharv6f/kv-cache-basics, accessed: 2026-05-01.

Hugging Face (2025) Llama 2 model documentation. https://huggingface.co/docs/transformers/model_doc/llama2, accessed: 2026-05-01.

NVIDIA (2026) Nvidia a100 tensor core gpu specifications. https://www.nvidia.com/en-eu/data-center/a100/, accessed: 2026-05-01.

Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2024) Splitwise: Efficient generative llm inference using phase splitting. in 2024 acm/ieee 51st annual international symposium on computer architecture (isca). *IEEE Computer Society, Los Alamitos, CA, USA* 118–132.

SGLang Team (2024) Sglang: Fast serving framework for large language models and vision language models. https://github.com/sgl-project/sglang, gitHub repository.

Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.